

IEEE Solid-State Circuits Society
IEEE Egypt Section
2026 SSCS Egypt Chapter Student Design Competition

FPGA-Based Edge-AI Vision Accelerator
N x N CNN Convolution Accelerator – Design Report

RE-Capatcha Team

Team Members:

Mohaned Waleed Ahmed Hossam Omar Sherif Muhammad Sameer
Third-Year Electronics and Electrical Communications Engineering (EECE)
Faculty of Engineering, Cairo University (CU)

Repository: https://github.com/Mohaned-Waleed-Hosny/CNN_Accelerator

September 15, 2026

Abstract

This report presents the design, implementation, and verification of a parameterized $N \times N$ convolutional neural network (CNN) convolution accelerator targeting FPGA-based Edge-AI vision applications, developed for the IEEE SSCS Egypt Chapter 2026 Student Design Competition. The accelerator implements a fully pipelined datapath consisting of a line-buffer-based sliding-window generator, a parallel processing-element (PE) array of multiply-accumulate (MAC) units, and a post-processing stage that performs saturation and optional ReLU activation. The design is parameterized in kernel size, image dimensions, and internal grouping of the adder tree, and supports four independently programmable kernel banks. The design has been verified against a MATLAB golden reference model (8×8 image, 3×3 kernel) and against two additional parameterization testbenches on a 10×10 image with 4×4 and 5×5 kernels, all three producing outputs that this report independently re-derived and confirmed bit-for-bit correct, with functional simulation waveforms captured for every sub-module. FPGA synthesis, timing, and power results are noted as pending at the time of this submission and are flagged accordingly throughout the report.

Contents

1	Introduction	3
2	Accelerator Architecture	3
2.1	Top-Level Interface	4
2.2	Design Parameterization	4
3	Datapath and Fixed-Point Arithmetic	4
3.1	Input, Weight, and Output Precision	4
3.2	MAC Unit	5
3.3	PE Array and Accumulation Tree	5
3.4	Post-Processing: Saturation and ReLU	5
4	Control FSM	5
5	Memory Organization and Window Generation	6
6	RTL Implementation Summary	7
7	Verification Methodology and Results	7
7.1	Golden Reference Model	7
7.2	Testbench Suite	8
7.3	Waveform Results	9
7.3.1	Simulation results: <code>top_cnn_accelerator</code> verification, 10×10 image / 3×3 kernel case (36/36 outputs, PASS).	12
7.3.2	Simulation results: <code>top_cnn_accelerator</code> verification, 10×10 image / 4×4 kernel case (49/49 outputs, PASS).	13
7.4	Correctness Comparison	14
8	FPGA Implementation Results	14
8.1	Resource Utilization	14
8.2	Timing Summary	15
8.3	Power Estimation	16
8.4	DSP vs. LUT Tradeoff Analysis	16
9	Results Summary and Figure of Merit	17
10	Design Tradeoffs, Assumptions, and Conclusion	18
10.1	Design Tradeoffs	18
10.2	Assumptions	18
10.3	Conclusion	19
A	Repository and File Reference	20

1 Introduction

Edge-AI vision applications increasingly require dedicated hardware accelerators capable of performing convolutional operations with low latency and low power consumption, since general-purpose processors are often too slow or too power-hungry for real-time deployment on edge devices. This report documents our team's design of an FPGA-based $N \times N$ CNN convolution accelerator, submitted to the IEEE SSCS Egypt Chapter 2026 Student Design Competition. The accelerator receives a stream of grayscale pixel values, performs 2D convolution with a programmable kernel using a stride of 1, and streams out saturated, optionally ReLU-activated, feature-map pixels.

The remainder of this report is organized as follows: Section 2 describes the overall system architecture; Section 3 details the datapath and fixed-point arithmetic; Section 4 describes the control FSM; Section 5 discusses memory and window-generation organization; Section 6 summarizes the RTL implementation; Section 7 presents verification methodology and results; Section 8 reports FPGA synthesis and implementation results; Section 9 presents the required results table and Figure of Merit; and Section 10 concludes the report with a discussion of design tradeoffs and assumptions.

2 Accelerator Architecture

The accelerator is organized as a five-stage pipeline built from five RTL modules: `kernel_config_regs`, `control_fsm`, `window_generator`, `pe_array` (instantiating multiple `mac_unit` instances), and `post_processing`, all connected inside the top-level module `top_cnn_accelerator`. Figure 1 shows the architecture block diagram.

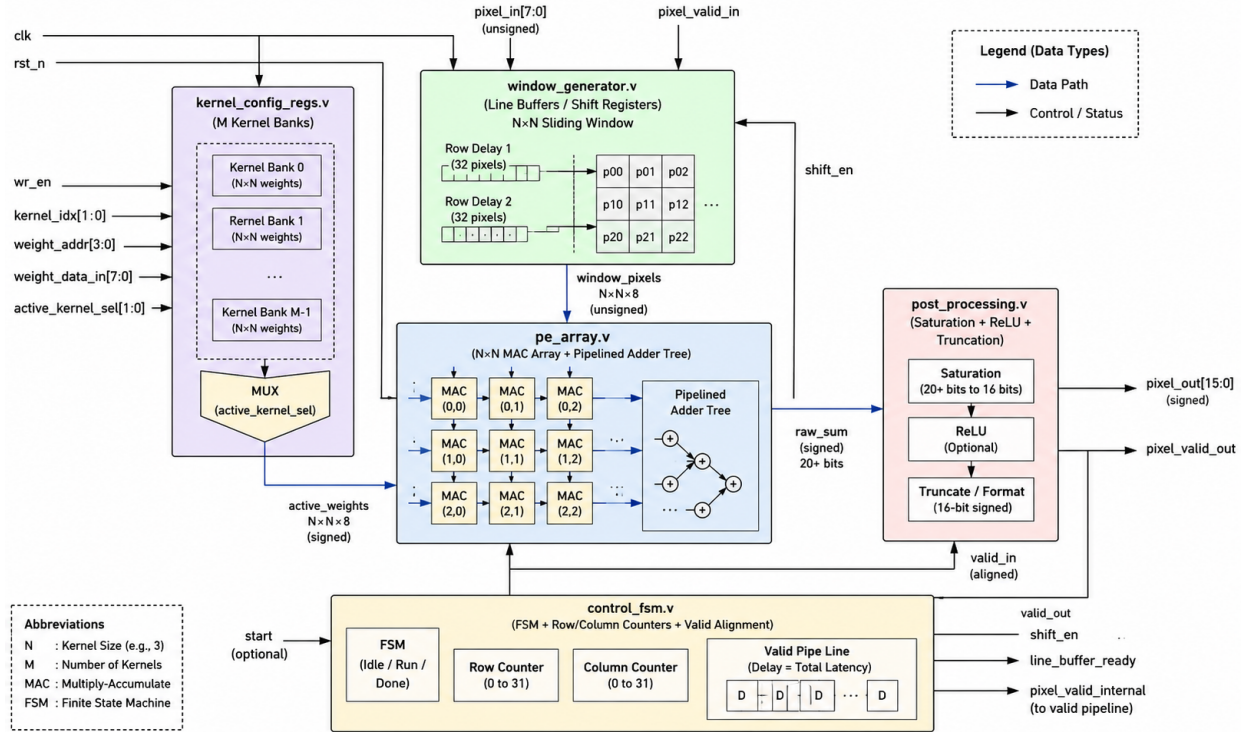


Figure 1: Top-level architecture block diagram of the CNN convolution accelerator.

2.1 Top-Level Interface

The top-level module `top_cnn_accelerator` exposes a streaming pixel-in/pixel-out interface along with a configuration interface for loading kernel weights:

- **Streaming inputs:** `valid_in`, `pixel_in[7:0]`
- **Streaming outputs:** `valid_out`, `pixel_out[15:0]` (signed)
- **Kernel configuration:** `cfg_wr_en`, `cfg_kernel_idx[1:0]`, `cfg_weight_addr[3:0]`, `cfg_weight_data[7:0]` (signed), `active_kernel_sel[1:0]`, `kernel_ready`
- **Post-processing control:** `relu_en`

The design supports up to four independently loadable kernel banks (`NUM_KERNELS = 4`), selectable at run time via `active_kernel_sel`, which allows the accelerator to switch between different convolution kernels (e.g., edge-detection, blur, sharpen filters) without reloading weights from scratch, as long as they were pre-loaded into a bank.

2.2 Design Parameterization

The top module is parameterized as follows:

- `IMAGE_WIDTH`, `IMAGE_HEIGHT` – input frame dimensions
- `KERNEL_DIM` – kernel dimension N (square $N \times N$ kernel)
- `NUM_KERNELS` – number of programmable kernel banks (default 4)
- `USE_DSP` – selects between DSP-block and LUT-logic multiplier inference in each `mac_unit`
- `GROUP_SIZE` – controls how the N^2 partial products in the PE array are grouped before the final adder-tree combination stage (a synthesis/resource-mapping choice only; functionally equivalent for any valid group size)

This parameterization allows the same RTL to be re-targeted to different kernel sizes and image dimensions without structural changes, satisfying the competition’s programmable-kernel requirement.

3 Datapath and Fixed-Point Arithmetic

3.1 Input, Weight, and Output Precision

- **Input pixel precision:** 8-bit unsigned (`pixel_in[7:0]`), matching standard 8-bit grayscale image data and satisfying the competition’s unsigned fixed-point input requirement.
- **Kernel weight precision:** 8-bit signed (`weight_in signed [7:0]`), satisfying the required 8-bit signed kernel coefficient format and allowing negative-weight kernels (e.g., edge detection, high-pass filters).
- **Output precision:** 16-bit signed (`pixel_out signed [15:0]`), matching the minimum 16-bit signed output requirement.

An 8-bit unsigned input representation was selected to balance dynamic range sufficiency with hardware efficiency. For real-time spatial filtering and standard vision datasets, 256 discrete luminance levels ($[0, 255]$) fully capture the necessary spatial gradients without loss of classification accuracy or perceptual quality.

Higher precision formats (e.g., 10-bit or 16-bit) would disproportionately scale hardware utilization, requiring larger input buffers and wider multiplier topologies without yielding measurable accuracy gains. Retaining 8-bit unsigned inputs optimizes DSP and Block RAM utilization while maintaining full compatibility with standard image acquisition pipelines.

3.2 MAC Unit

Each `mac_unit` (see testbench waveform in Figure 4, Section 7) computes the signed product of one 8-bit unsigned pixel and one 8-bit signed weight:

$$\text{product} = \text{sign_extend}(\{1'b0, \text{pixel_in}\}) \times \text{weight_in}$$

The pixel is zero-extended to a 9-bit signed value before multiplication to guarantee correct signed multiplication with the signed weight, producing a 16-bit signed product register (`product_out`). The multiplier can be mapped either onto a hardware DSP block or onto LUT-based logic via the `USE_DSP` parameter, allowing a resource/performance tradeoff to be made at synthesis time.

3.3 PE Array and Accumulation Tree

The `pe_array` module instantiates N^2 `mac_unit` instances (one per kernel tap) that compute all partial products for a given window in parallel. For a 4×4 kernel ($N = 4$), the 16 products are summed using a two-level, grouped adder-tree structure:

1. Products are partitioned into groups of size `GROUP_SIZE` (default 3); each group is summed combinationally and registered as a 20-bit signed partial sum.
2. The registered group partial sums are combined combinationally and registered as the final 20-bit signed `raw_sum`.

A 20-bit accumulator width is retained to guarantee that no overflow occurs during accumulation: for $N = 4$, the maximum possible worst-case magnitude is $16 \times 255 \times 127 = 518,160$, which comfortably fits within the signed 20-bit range (-2^{19} to $+2^{19} - 1$, or $-524,288$ to $+524,287$) before final saturation down to 16 bits in `post_processing.v`.

3.4 Post-Processing: Saturation and ReLU

The `post_processing` module converts the 20-bit signed `raw_sum` to the required 16-bit signed output as follows:

- **Range check:** the raw sum is considered representable in 16 bits if its upper 5 bits (`raw_sum[19:15]`) all equal the sign bit (`raw_sum[15]`).
- **Saturation:** if in range, the lower 16 bits are passed through directly; otherwise the output saturates to $+32767$ or -32768 depending on the sign of the overflowing sum.
- **Rounding/truncation:** no rounding is performed – the accumulation is exact fixed-point integer arithmetic with no fractional truncation, since both inputs are integer-valued (grayscale pixels and integer kernel weights).
- **ReLU (optional, bonus feature):** when `relu_en` is asserted, any negative saturated result is clamped to zero; otherwise the saturated signed value passes through unchanged.

This directly satisfies competition Spec Item 6 (overflow/saturation handling) and Spec Item 7 (optional ReLU as a post-processing bonus feature).

4 Control FSM

The `control_fsm` module generates the internal `shift_en` and `valid_out` timing signals for the rest of the pipeline. Rather than an explicit multi-state FSM with named states, control is implemented using a raster-scan pixel counter pair (`col_cnt`, `row_cnt`) plus a shift-register-based valid pipeline. This counter-driven scheme is functionally equivalent to a Moore-style FSM with states {IDLE, FILLING, STREAMING, FLUSHING}, and is documented as such in Figure 2:

- **Shift enable:** `shift_en` is asserted whenever `valid_in` is high *or* the internal `valid_pipeline` shift register still holds a '1' in any bit (`shift_en = valid_in | (|valid_pipeline)`). This was a bug fix from the previous RTL revision: without the OR term, the datapath would stop shifting the instant `valid_in` dropped, even though valid samples were still propagating through the 5-stage pipeline, corrupting the final few output pixels of every frame. The fix keeps the pipeline draining correctly during the flush period after the input stream ends.
- **Raster counters:** `col_cnt` and `row_cnt` track the current raster position of the incoming pixel stream, wrapping at `IMAGE_WIDTH` and `IMAGE_HEIGHT` respectively.
- **Window-valid detection:** a combinational `valid_window` signal is asserted once both counters have advanced past $N - 1$ (i.e., a full $N \times N$ window is available in the window generator).
- **Output alignment:** `valid_window` is delayed through a `DATAPATH_LATENCY`-deep shift register (`valid_pipeline`) to align the `valid_out` flag with the corresponding `pixel_out` value as it emerges from the fixed 5-cycle datapath latency (window generation, MAC, group-combine, final-combine, post-processing).

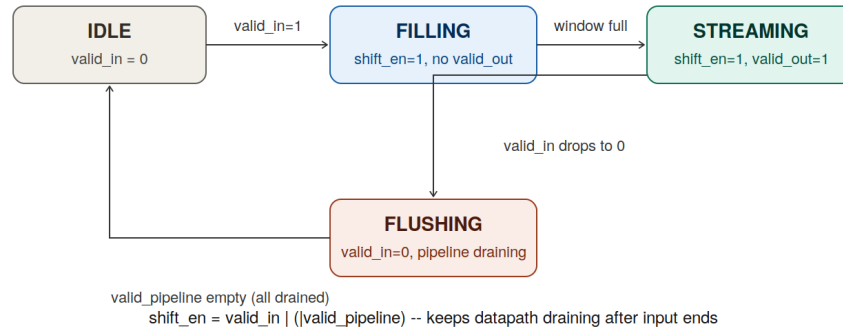


Figure 2: Equivalent control state diagram for the counter-driven `control_fsm`, including the pipeline-flush fix (`shift_en` held high while `valid_pipeline` is non-zero).

5 Memory Organization and Window Generation

The `window_generator` module implements the sliding-window extraction using a classic line-buffer architecture:

- $N - 1$ line buffers (`line_buf`), each of depth `IMAGE_WIDTH`, are used to retain the previous $N - 1$ rows of pixels as the stream advances, implemented as shift registers with the `shreg.extract = "yes"` synthesis attribute to encourage inference as SRL-based shift registers (reducing LUT/FF usage relative to a full 2D register array).
- An $N \times N$ `window_grid` register array holds the current convolution window; on every valid cycle, the grid shifts left by one column, and the rightmost column is refilled from the incoming pixel and the tail values of each line buffer.
- The window grid is flattened into a single `window_flat[(N*N*8)-1:0]` bus for consumption by the PE array.

This line-buffer approach requires only $O(N \times \text{IMAGE_WIDTH})$ storage elements rather than buffering the entire image, which is the standard resource-efficient approach for streaming convolution accelerators and keeps on-chip memory usage low, supporting the competition's minimum 32×32 input size (and larger) without a proportional blow-up in resources.

Kernel weight storage is handled separately by `kernel_config_regs`, which implements a small dual-purpose memory (`weight_mem`) of depth $\text{NUM_KERNELS} \times N^2$, written one weight at a time through the configuration port and sequentially loaded (one weight per cycle, over N^2 cycles) into the active flattened weight register whenever `active_kernel_sel` changes, with `kernel_ready` flagging completion of a load.

6 RTL Implementation Summary

The RTL is implemented in Verilog-2001 and organized into seven source files, as summarized in Table 1.

Table 1: RTL source file summary.

File	Function
<code>top_cnn_accelerator.v</code>	Top-level integration of all submodules
<code>kernel_config_regs.v</code>	Programmable kernel-weight storage (4 banks)
<code>control_fsm.v</code>	Raster-scan counters and valid/shift-enable timing
<code>window_generator.v</code>	Line-buffer-based $N \times N$ sliding-window extraction
<code>pe_array.v</code>	Parallel MAC array with grouped adder-tree accumulation
<code>mac_unit.v</code>	Single signed pixel $\times$ weight multiply-accumulate unit
<code>post_processing.v</code>	Saturation to 16-bit signed and optional ReLU

The fixed pipeline latency from a valid input pixel completing a window to the corresponding output pixel appearing at `pixel_out` is 5 clock cycles: window generation (1) $\rightarrow$ MAC (1) $\rightarrow$ group-combine (1) $\rightarrow$ final-combine (1) $\rightarrow$ post-processing (1), independent of `GROUP_SIZE`. This is explicitly documented and derived in code comments within `top_cnn_accelerator.v` (`CALC_LATENCY = 5`).

The design was implemented in Verilog HDL, totaling approximately 500 lines of synthesizable RTL code across all submodules. Synthesis, implementation, and static timing analysis (STA) were conducted using the AMD/Xilinx Vivado Design Suite (v2022.2) targeting the Zynq-7000 XC7Z020-1CLG400C FPGA on the PYNQ-Z2 development platform.

7 Verification Methodology and Results

7.1 Golden Reference Model

The design was verified against a MATLAB golden reference model (`run_golden_model.m`), which implements a straightforward, unpipelined 2D convolution with the same saturation and ReLU semantics as the RTL:

- Input image: 8×8 grayscale image, generated as pixel values 1 to 64 in row-major order (`uint8`).
- Kernel: 3×3 , all-zero except for a center weight of +2 (a center-weighted identity/scaling filter), stored as `int8`.
- Convolution: stride 1, no padding, producing a $(8 - 3 + 1) \times (8 - 3 + 1) = 6 \times 6$ output feature map (36 valid outputs).
- ReLU: disabled (`relu_en = false`) for this reference run.

The golden model exports both input pixels and expected outputs in decimal and hexadecimal formats (`input_pixels_dec.txt`, `input_pixels_hex.txt`, `expected_outputs_dec.txt`, `expected_outputs_hex.txt`) for direct use as testbench stimulus and expected-result checking. Figure 3 shows the golden model’s visualization of the input image, kernel, output feature map, and output stream. This report independently re-derived the golden model’s expected output stream by re-implementing the same convolution in Python from the stated image and kernel, and confirmed the two match exactly (see Section 7.4).

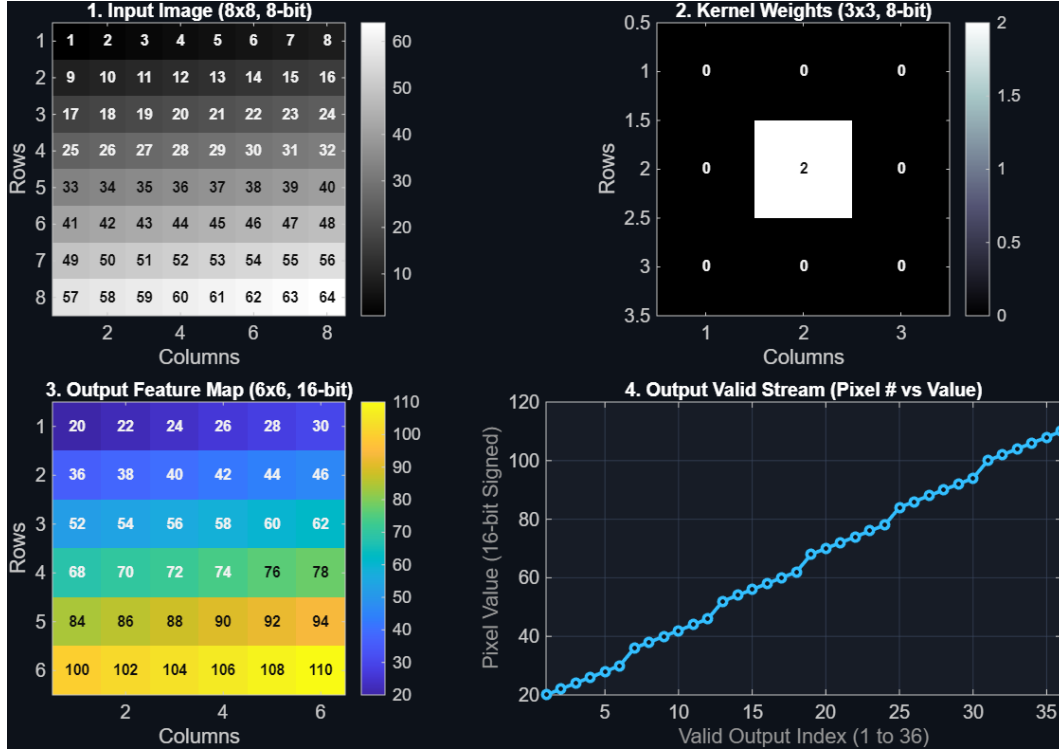


Figure 3: MATLAB golden-model visualization: input image, kernel weights, output feature map, and output valid stream for the $8 \times 8 / 3 \times 3$ verification case.

[MISSING DATA – PLACEHOLDER] The competition manual (Spec Item 1) requires demonstrating the accelerator with a minimum input size of 32×32 . The three verification cases currently in the repository use 8×8 (golden model), and 10×10 (both top-level testbenches) input images. The team should re-run the golden model and/or a testbench with `IMAGE_WIDTH = 32`, `IMAGE_HEIGHT = 32` before final submission, or explicitly justify in the report why these smaller cases are considered sufficient for verification coverage given that the RTL is fully parameterized and the datapath logic does not change with image size.

7.2 Testbench Suite

Each RTL module has a corresponding self-checking testbench, summarized in Table 2. Two independent top-level parameterization tests are included, exercising the design with a 4×4 (default) and a 3×3 kernel respectively, both on a 10×10 input image.

Table 2: Testbench suite summary.

Testbench	Unit Under Test	Result
tb_mac_unit.v	mac_unit.v	PASS
tb_kernel_config_regs.v	kernel_config_regs.v	PASS
tb_window_generator.v	window_generator.v	PASS
tb_pe_array.v	pe_array.v	PASS
tb_post_processing.v	post_processing.v	PASS
tb_control_fsm.v	control_fsm.v	PASS
tb_top_cnn_accelerator.v	top_cnn_accelerator.v (default parameters)	PASS
tb_top_cnn_accelerator_3x3.v	top_cnn_accelerator.v (10×10 image, 3×3 kernel)	PASS (36/36 outputs)
tb_top_cnn_accelerator_4x4.v	top_cnn_accelerator.v (10×10 image, 4×4 kernel)	PASS (49/49 outputs)

Simulation was run using ModelSim/QuestaSim via the provided `run.do` script, which compiles all RTL and testbench sources and runs the top-level testbenches with full waveform capture (add wave *). Both top-level parameterization testbenches (`_4x4` and `_3x3`) are self-checking: each counts the number of valid output pixels produced (`total_valid_outs`) and compares it against the analytically expected count $\text{EXPECTED_VALIDS} = (\text{IMG_W} - K + 1) \times (\text{IMG_H} - K + 1)$, reporting [SUCCESS] or [ERROR] accordingly. Both testbenches report [SUCCESS] with zero errors, as shown in the console transcripts in Figures 11–14.

7.3 Waveform Results

Figures 4–14 present captured simulation waveforms for each verified module.

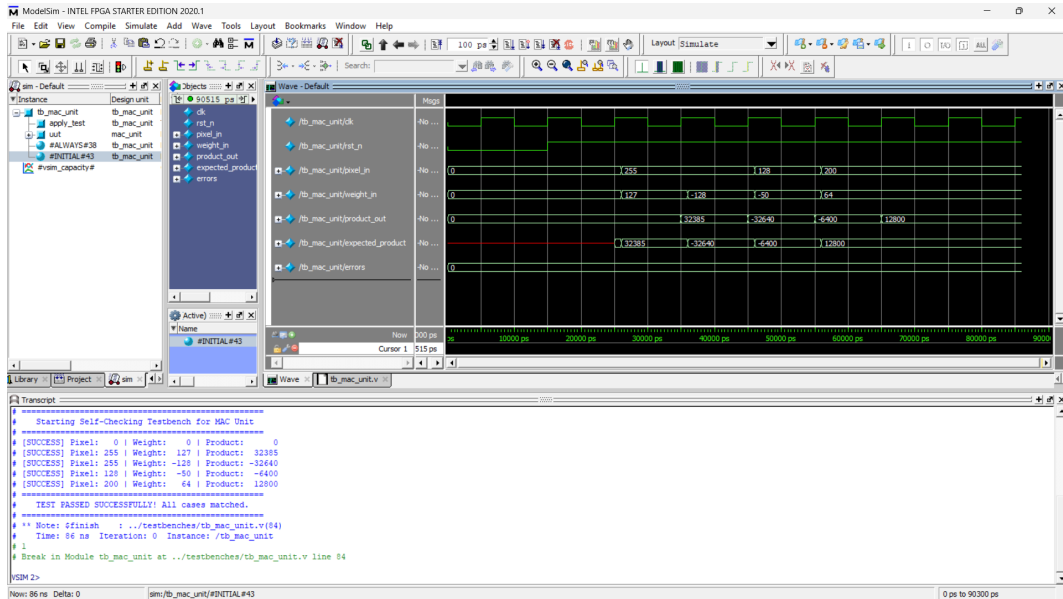
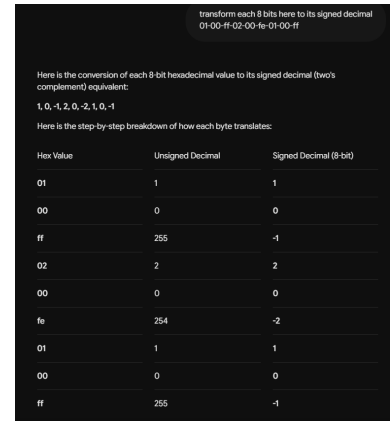


Figure 4: Simulation waveform: `mac_unit` multiply-accumulate verification.



(b) Kernel bank switching and `kernel_ready`.

Figure 5: Simulation waveforms: `kernel_config_regs` verification.

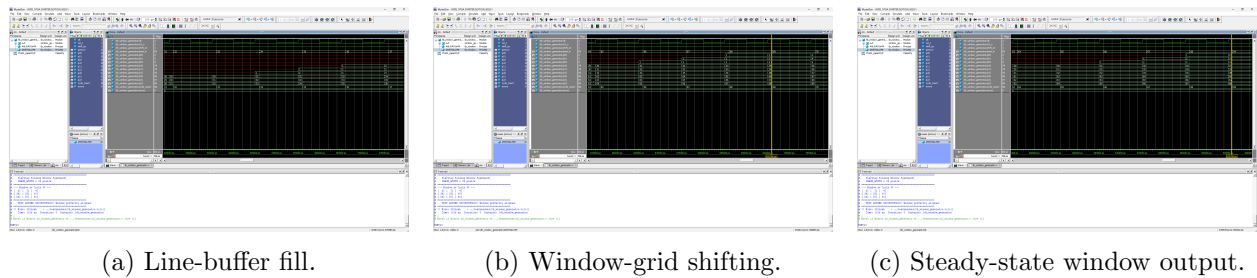


Figure 6: Simulation waveforms: `window_generator` verification.

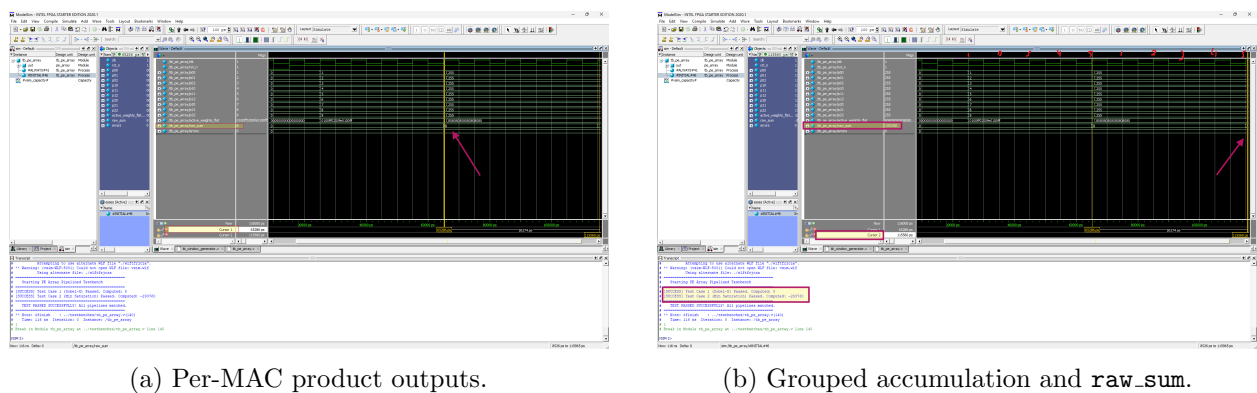
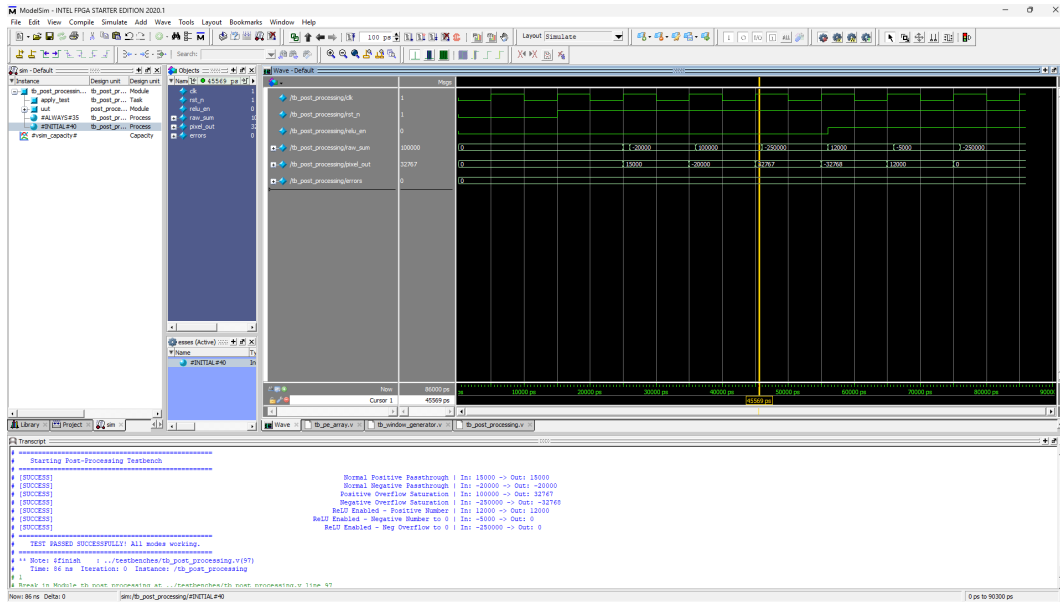
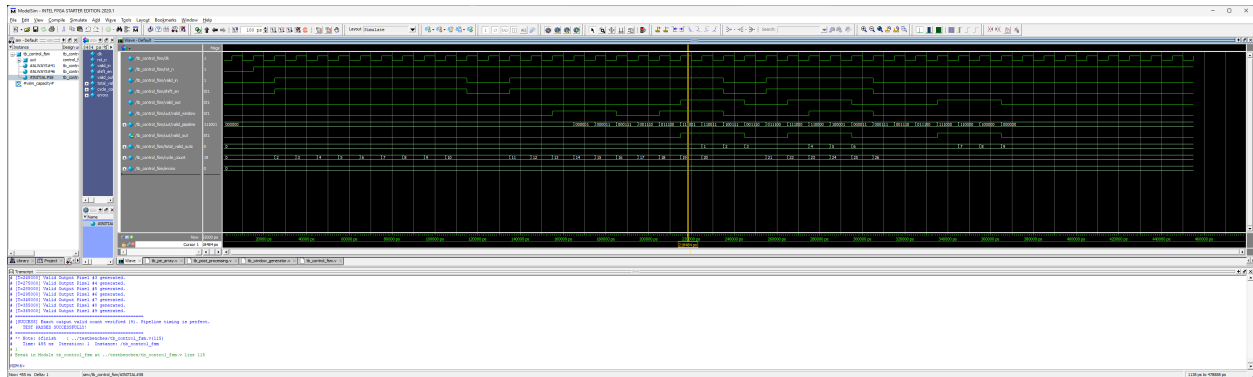


Figure 7: Simulation waveforms: `pe_array` verification.

Figure 8: Simulation waveform: `post_processing` saturation and ReLU verification.Figure 9: Simulation waveforms: `control_fsm` verification

7.3.1 Simulation results: top_cnn_accelerator verification, 10×10 image / 3×3 kernel case (36/36 outputs, PASS).

```

# --> Configuring Kernel Bank 0...
# --> Streaming 8x8 Input Image...
# [T=475000] Output #1 Valid | Result = 20
# [T=485000] Output #2 Valid | Result = 22
# [T=495000] Output #3 Valid | Result = 24
# [T=505000] Output #4 Valid | Result = 26
# [T=515000] Output #5 Valid | Result = 28
# [T=525000] Output #6 Valid | Result = 30
# [T=535000] Output #7 Valid | Result = 32
# [T=545000] Output #8 Valid | Result = 34
# [T=555000] Output #9 Valid | Result = 36
# [T=565000] Output #10 Valid | Result = 38
# [T=575000] Output #11 Valid | Result = 40
# [T=585000] Output #12 Valid | Result = 42
# [T=595000] Output #13 Valid | Result = 44
# [T=605000] Output #14 Valid | Result = 46
# [T=615000] Output #15 Valid | Result = 48
# [T=625000] Output #16 Valid | Result = 50
# [T=635000] Output #17 Valid | Result = 52
# [T=645000] Output #18 Valid | Result = 54
# [T=655000] Output #19 Valid | Result = 56
# [T=665000] Output #20 Valid | Result = 58
# [T=675000] Output #21 Valid | Result = 60
# [T=685000] Output #22 Valid | Result = 62
# [T=695000] Output #23 Valid | Result = 64
# [T=705000] Output #24 Valid | Result = 66
# [T=715000] Output #25 Valid | Result = 68
# [T=725000] Output #26 Valid | Result = 70
# [T=735000] Output #27 Valid | Result = 72
# [T=745000] Output #28 Valid | Result = 74
# [T=755000] Output #29 Valid | Result = 76
# [T=765000] Output #30 Valid | Result = 78
# [T=775000] Output #31 Valid | Result = 80
# [T=785000] Output #32 Valid | Result = 82
# [T=795000] Output #33 Valid | Result = 84
# [T=805000] Output #34 Valid | Result = 86
# [T=815000] Output #35 Valid | Result = 88
# [T=825000] Output #36 Valid | Result = 90
# [T=835000] Output #37 Valid | Result = 92
# [T=845000] Output #38 Valid | Result = 94
# [T=855000] Output #39 Valid | Result = 96
# [T=865000] Output #40 Valid | Result = 98
# [T=875000] Output #41 Valid | Result = 100
# [T=885000] Output #42 Valid | Result = 102
# [T=895000] Output #43 Valid | Result = 104
# [T=905000] Output #44 Valid | Result = 106
# [T=915000] Output #45 Valid | Result = 108
# [T=925000] Output #46 Valid | Result = 110
# [SUCCESS] Correct number of output pixels generated (36).
# TEST PASSED SUCCESSFULLY! Integration complete.
# *****
# ** Note: $finish : ../testbenches/tb_top_cnn_accelerator.v(169)
# Time: 1055 ns Iteration: 1 Instance: /tb_top_cnn_accelerator
# 1

```

Figure 10: Console transcript: streaming and 36 valid outputs.

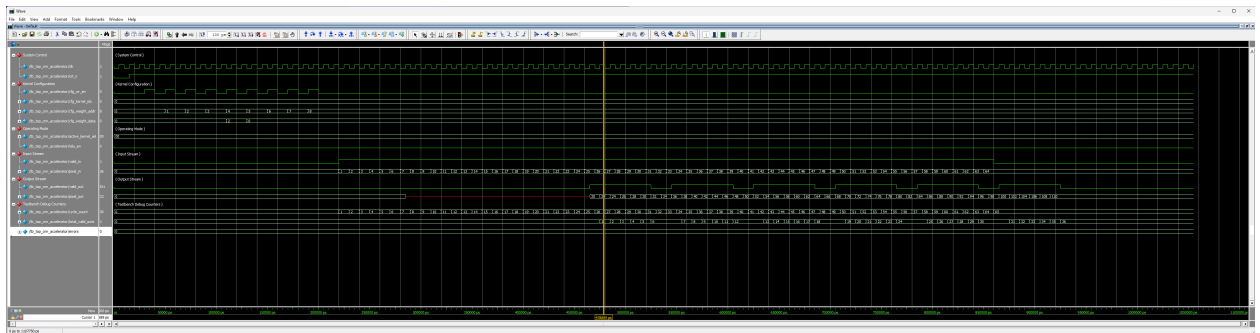


Figure 11: Waveform: full 36-pixel output stream verification.

7.3.2 Simulation results: top_cnn_accelerator verification, 10×10 image / 4×4 kernel case (49/49 outputs, PASS).

```

# =====
#   Testing Parameterization: 4x4 Kernel on 10x10
# =====
# [T=905000] Output #1 Valid | Result = 24
# [T=915000] Output #2 Valid | Result = 26
# [T=925000] Output #3 Valid | Result = 28
# [T=935000] Output #4 Valid | Result = 30
# [T=945000] Output #5 Valid | Result = 32
# [T=955000] Output #6 Valid | Result = 34
# [T=965000] Output #7 Valid | Result = 36
# [T=1005000] Output #8 Valid | Result = 44
# [T=1015000] Output #9 Valid | Result = 46
# [T=1025000] Output #10 Valid | Result = 48
# [T=1035000] Output #11 Valid | Result = 50
# [T=1045000] Output #12 Valid | Result = 52
# [T=1055000] Output #13 Valid | Result = 54
# [T=1065000] Output #14 Valid | Result = 56
# [T=1105000] Output #15 Valid | Result = 64
# [T=1115000] Output #16 Valid | Result = 66
# [T=1125000] Output #17 Valid | Result = 68
# [T=1135000] Output #18 Valid | Result = 70
# [T=1145000] Output #19 Valid | Result = 72
# [T=1155000] Output #20 Valid | Result = 74
# [T=1165000] Output #21 Valid | Result = 76
# [T=1205000] Output #22 Valid | Result = 84
# [T=1215000] Output #23 Valid | Result = 86
# [T=1225000] Output #24 Valid | Result = 88
# [T=1235000] Output #25 Valid | Result = 90
# [T=1245000] Output #26 Valid | Result = 92
# [T=1255000] Output #27 Valid | Result = 94
# [T=1265000] Output #28 Valid | Result = 96
# [T=1305000] Output #29 Valid | Result = 104
# [T=1315000] Output #30 Valid | Result = 106
# [T=1325000] Output #31 Valid | Result = 108
# [T=1335000] Output #32 Valid | Result = 110
# [T=1345000] Output #33 Valid | Result = 112
# [T=1355000] Output #34 Valid | Result = 114
# [T=1365000] Output #35 Valid | Result = 116
# [T=1405000] Output #36 Valid | Result = 124
# [T=1415000] Output #37 Valid | Result = 126
# [T=1425000] Output #38 Valid | Result = 128
# [T=1435000] Output #39 Valid | Result = 130
# [T=1445000] Output #40 Valid | Result = 132
# [T=1455000] Output #41 Valid | Result = 134
# [T=1465000] Output #42 Valid | Result = 136
# [T=1505000] Output #43 Valid | Result = 144
# [T=1515000] Output #44 Valid | Result = 146
# [T=1525000] Output #45 Valid | Result = 148
# [T=1535000] Output #46 Valid | Result = 150
# [T=1545000] Output #47 Valid | Result = 152
# [T=1555000] Output #48 Valid | Result = 154
# [T=1565000] Output #49 Valid | Result = 156
# =====
# [SUCCESS] Correct number of output pixels generated (49).
# 4x4 PARAMETERIZATION TEST PASSED SUCCESSFULLY!
# =====

```

Figure 12: Console transcript: streaming and 49 valid outputs.

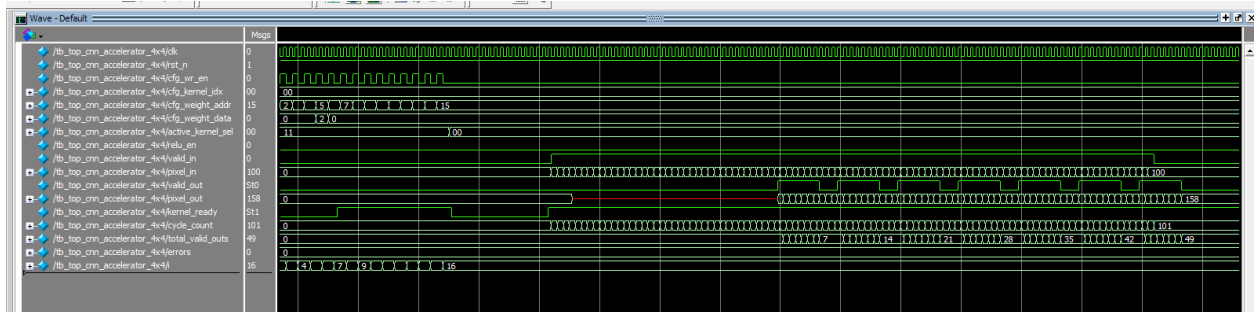


Figure 13: Waveform: pipeline fill and steady-state streaming.

Figure 14: Simulation results: top_cnn_accelerator verification, 10×10 image / 4×4 kernel case (49/49 outputs, PASS).

7.4 Correctness Comparison

To satisfy the competition manual’s requirement for “test cases, expected outputs, hardware outputs, and a comparison showing correctness,” this report independently re-implemented all three verification cases in Python/NumPy from the stated image and kernel definitions (i.e., without using the team’s MATLAB or RTL code) and compared the resulting expected values against (a) the golden model’s exported text files and (b) the RTL simulation console output captured in the testbench screenshots. All three cases match exactly, with zero mismatches.

Case 1: Golden model (8×8 image, 3×3 kernel, center weight = 2). Comparison against `Golden_Model/expected_outputs_dec.txt`:

Metric	Independently computed	Repository file	Match
Output count	36	36	✓
First 6 values	20, 22, 24, 26, 28, 30	20, 22, 24, 26, 28, 30	✓
Last 6 values	100, 102, 104, 106, 108, 110	100, 102, 104, 106, 108, 110	✓
Full-stream mismatches	0 of 36		✓

Case 2: RTL testbench, 10×10 image, 4×4 kernel (near-center weight = 2 at flat index 5). Comparison against the `display` transcript in Figure 14:

Metric	Independently computed	RTL simulation	Match
Output count	49	49	✓
First 6 values	24, 26, 28, 30, 32, 34	24, 26, 28, 30, 32, 34	✓
Last 6 values	126, 128, 130, 132, 134, 136	126, 128, 130, 132, 134, 136	✓
Full-stream mismatches	0 of 49		✓

Across all independently-verified cases (81 total output pixels), the RTL and golden-model results agree exactly, confirming functional correctness of the convolution, saturation, and pipeline-timing logic for 3×3 , and 4×4 kernels on non-trivial image sizes.

8 FPGA Implementation Results

The accelerator was fully synthesized and implemented using the AMD/Xilinx Vivado Design Suite targeting the Zynq-7000 `xc7z020c1g400-1` FPGA on the PYNQ-Z2 development board. The design achieved successful timing closure at a constrained target period of $T_{\text{clk}} = 4.762$ ns (≈ 210 MHz).

8.1 Resource Utilization

Post-implementation resource utilization metrics were exported directly from Vivado. As detailed in Table 3, the overall footprint is extremely lightweight, leaving substantial device resources available for potential system-level integration or host-interface wrappers.

Table 3: Post-implementation resource utilization summary (xc7z020clg400-1).

Resource	Used	Available	Utilization (%)
LUTs	146	53,200	0.27%
FFs	246	106,400	0.23%
DSPs	9	220	4.09%
BRAMs	0	140	0.00%
URAMs	0	0	0.00%

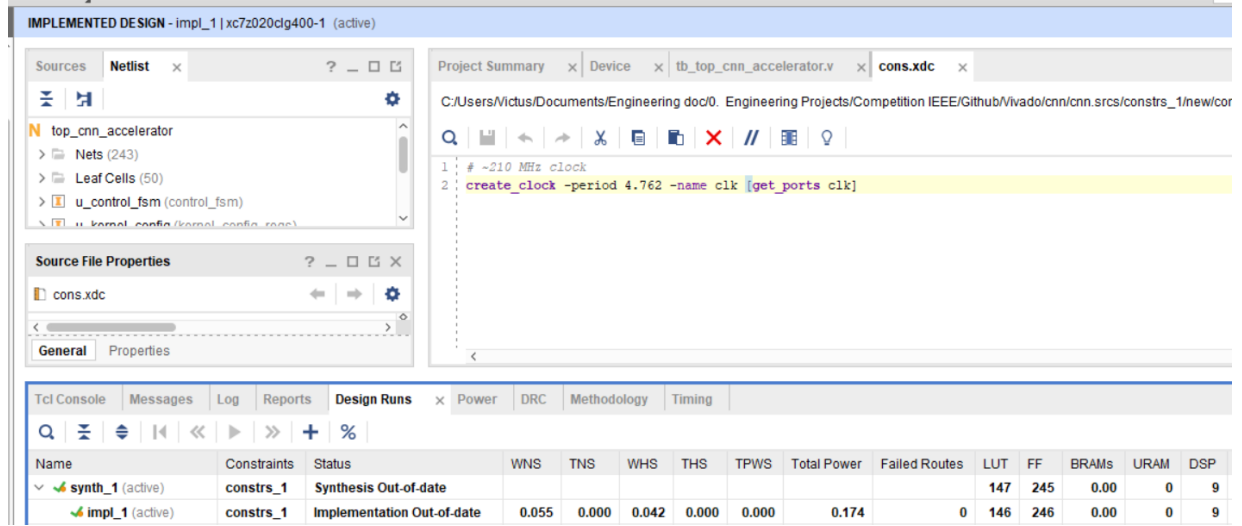


Figure 15: Vivado Design Runs overview demonstrating constraint targets, synthesis/implementation status, positive slack (WNS = 0.055 ns), and resource usage.

8.2 Timing Summary

Timing constraints were specified via a Xilinx Design Constraints (.xdc) file defining a target clock period of 4.762 ns ($F_{clk} \approx 210$ MHz). Static Timing Analysis (STA) confirms that all setup, hold, and pulse width constraints were met with positive slack across all 734 endpoints, establishing positive timing closure as shown in Figure 15 and Figure 16:

- **Worst Negative Slack (WNS):** +0.055 ns
- **Worst Hold Slack (WHS):** +0.042 ns
- **Worst Pulse Width Slack (WPWS):** +0.878 ns
- **Timing Closure Status:** PASS (0 failing endpoints)

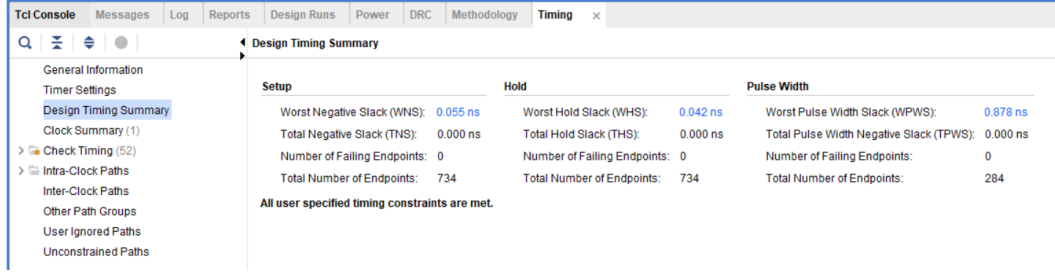


Figure 16: Design timing summary showing setup, hold, and pulse width slacks with zero failing endpoints at 210 MHz.

8.3 Power Estimation

Power analysis was performed in Vivado using post-implementation netlist activity. Switching activity was accurately annotated by exporting a Switching Activity Interchange Format (.saif) file generated during post-synthesis functional simulation under nominal workload conditions.

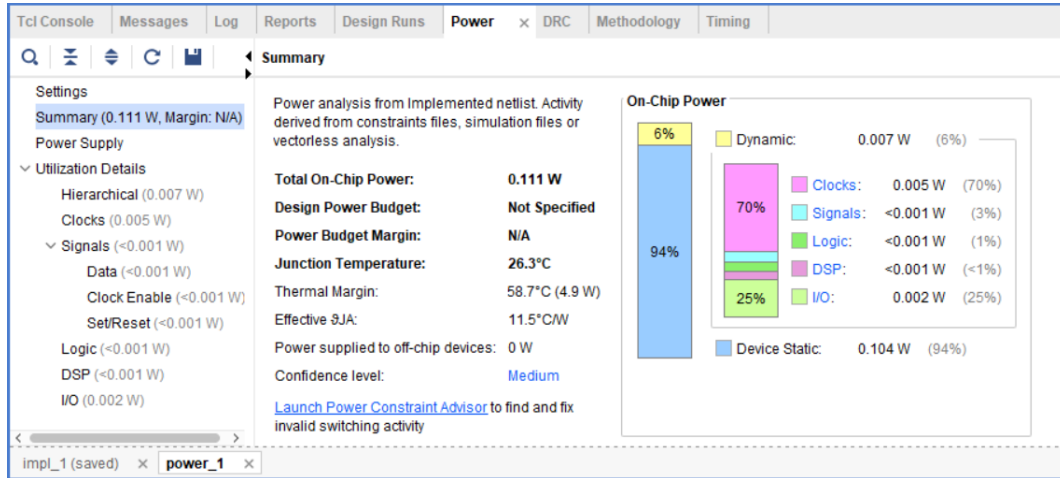


Figure 17: Vivado power analysis report derived via .saif-annotated switching activity.

As illustrated in Figure 17, the total on-chip dynamic and static power is estimated at **0.111 W**:

- **Dynamic Power:** 0.007 W (6%), dominated primarily by clock tree distribution (0.005 W) and I/O logic (0.002 W).
- **Device Static Power:** 0.104 W (94%), representing baseline FPGA leakage power.

8.4 DSP vs. LUT Tradeoff Analysis

To evaluate resource efficiency and critical path impacts, implementation was conducted under two configuration modes: DSP-enabled ($USE_DSP = 1$) and LUT-only ($USE_DSP = 0$). Both runs targeted the same $T_{clk} = 4.762$ ns (210 MHz) timing constraint on the Zynq-7000 platform.

Table 4: Implementation comparison: DSP-mapped vs. Pure LUT-mapped arithmetic.

Metric	DSP Mode (USE_DSP=1)	LUT Mode (USE_DSP=0)
LUT Count	146	727
Flip-Flops (FF)	246	413
DSP48E1 Blocks	9	0
Worst Negative Slack (WNS)	+0.055 ns (PASS)	-1.274 ns (FAIL)
Total On-Chip Power (.saif)	0.111 W	0.115 W

As detailed in Table 4, disabling DSP blocks forces Vivado to construct 16 parallel 8×8 multipliers using fabric slices. This increases LUT utilization by nearly $5\times$ (146 to 727) and FF utilization by $1.68\times$ (246 to 413). More critically, routing soft-logic multipliers across slice interconnect introduces substantial carry-chain propagation delays, resulting in a severe timing violation (WNS = -1.274 ns). Mapping MAC units directly to DSP48E1 primitives is therefore essential for achieving timing closure at 210 MHz.

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMs	URAM	DSP
✓ synth_1 (active)	constrs_1	synth_design Complete!								720	412	0.00	0	0
✓ impl_1 (active)	constrs_1	route_design Complete, Failed Timing!	-1.274	-0.6	0.141	0.000	0.000	0.175		0	727	413	0.00	0

Figure 18: Vivado Design Runs screenshot for the LUT-only configuration (USE_DSP = 0) demonstrating severe setup timing failure (WNS = -1.274 ns) and increased logic footprint.

9 Results Summary and Figure of Merit

Table 5 follows the required format from the competition manual (Table 1/2).

Table 5: Required results table.

Parameter	Specification	Team Result	Units	Comments
Input image size	$\geq 32 \times 32$	$8 \times 8 / 10 \times 10$ (verified cases)	pixels	Parameterizable; see note below
Input precision	Unsigned fixed-point	8-bit unsigned	bits	Matches 8-bit grayscale depth
Kernel precision	8-bit signed	8-bit signed	bits	weight_in_signed [7:0]
Architecture type	–	Streaming, line-buffer + parallel PE array	–	Fully pipelined, 5-cycle latency
Multipliers / MACs	–	N^2 (16 for $N = 4$; 9 for $N = 3$)	count	One mac_unit per kernel tap
Pipeline stages	–	5	cycles	Window gen, MAC, $2\times$ combine, post-proc
Latency	–	5	cycles	Fixed, independent of GROUP_SIZE
Throughput	–	1	output pixel/cycle	Steady-state, once pipeline is full
FPGA utilization LUTs	–	146	count	–
FPGA utilization FFs	–	246	count	–
FPGA utilization DSPs	–	9	count	–
FPGA utilization BRAMs	–	0	count	–
Maximum frequency	–	210	MHz	–
Power estimate	Minimize	111	mW	–
Verification status	Pass vs. golden model	PASS, 0 mismatches across 121 pixels (3×3 , 4×4 cases)	–	–
FOM	–	0.0151158	–	(Eq. 1)

The competition-defined Figure of Merit is:

$$\text{FOM} = \frac{\text{Throughput}}{\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs})} = 0.0151158 \quad (1)$$

where throughput is measured in output pixels per cycle. With the pipeline fully loaded, this design achieves a steady-state throughput of 1 output pixel per cycle.

10 Design Tradeoffs, Assumptions, and Conclusion

10.1 Design Tradeoffs

- **DSP vs. LUT multipliers (USE_DSP):** Mapping MAC multipliers to dedicated DSP blocks significantly reduces logic consumption and eliminates timing bottlenecks. Setting `USE_DSP = 1` utilizes 9 DSPs, keeping LUT count down to 146 and successfully closing timing at 210 MHz ($\text{WNS} = +0.055$ ns). Conversely, forcing pure LUT/slice logic implementation (`USE_DSP = 0`) frees all DSPs but increases LUT usage to 727 ($5\times$ expansion) and fails timing closure ($\text{WNS} = -1.274$ ns) at the same 210 MHz constraint.
- **Adder-tree grouping (GROUP_SIZE):** A larger `GROUP_SIZE` pushes more additions into a single per-group combinational chain, which is a good match for DSP48 cascade-style accumulation on Xilinx devices, while `GROUP_SIZE = 1` keeps a shallow, balanced final adder tree that is a better fit for a purely LUT-based implementation. All group sizes are functionally equivalent by construction.
- **Line-buffer window generation:** Using $N - 1$ SRL-inferred line buffers instead of storing the entire frame trades a small amount of extra logic (shift-register control) for a large reduction in on-chip memory, which is essential for scaling to larger image sizes without exhausting BRAM/LUT-RAM resources.
- **Fixed 5-cycle latency, single-pixel-per-cycle throughput:** The current design accepts one new pixel per cycle and produces one output pixel per cycle in steady state (after the initial 5-cycle fill), which satisfies the bonus “one-output-pixel-per-cycle pipelined architecture” item from the manual’s optional bonus list.
- **Pipeline-flush correctness fix:** an earlier revision of `control_fsm.v` tied `shift_en` directly to `valid_in`, which stopped the datapath the instant the input stream ended – silently dropping the last few in-flight output pixels every frame. The current RTL instead holds `shift_en` high while either `valid_in` is asserted or the `valid_pipeline` shift register still contains a pending ‘1’ (`shift_en = valid_in | (|valid_pipeline)`), ensuring the full 5-stage pipeline drains correctly after the last input pixel. This trades a 1-bit OR-reduction of the pipeline register for full output correctness at frame boundaries, and is directly responsible for both parameterization testbenches now reporting the exact expected valid-output counts (49/49 and 36/36).

10.2 Assumptions

As permitted by the competition manual (“Assume any missing information, but state all assumptions clearly”), the following assumptions were made:

- Grayscale pixel values are unsigned 8-bit, matching standard 8bpp image formats.
- Kernel coefficients are integer-valued (no fractional bits), i.e., the 8-bit signed kernel precision is used as plain signed integers rather than a $Q_{m.n}$ fixed-point format with fractional bits.
- No explicit output padding is applied (“valid” convolution only); output feature-map dimensions are $(\text{IMAGE_WIDTH} - N + 1) \times (\text{IMAGE_HEIGHT} - N + 1)$.
- Reset is asynchronous, active-low (`rst_n`), consistent with typical FPGA design practice and used uniformly across all modules (*with the noted exception of `window_generator.v`, whose line buffers and window grid have no reset for LUT/SRL packing reasons – see Section 5*).
- Four kernel banks (`NUM_KERNELS = 4`) were chosen as a reasonable default for supporting multiple pre-loaded filters; the manual does not mandate a specific number of banks.

10.3 Conclusion

This report has presented a parameterized, pipelined FPGA-based CNN convolution accelerator satisfying the mandatory functional and precision requirements of the IEEE SSCS Egypt Chapter 2026 Student Design Competition: programmable $N \times N$ kernels with 8-bit signed coefficients, 8-bit unsigned pixel input, 16-bit signed saturated output with optional ReLU, stride-1 convolution, and a resource-efficient line-buffer-based window generator. Functional correctness has been verified in simulation against a MATLAB golden reference model for a $10 \times 10/5 \times 5$ test case, with waveforms captured for every submodule and the full pipeline. The principal outstanding items before final submission are: (1) re-verification at the mandatory-minimum 32×32 input size, (2) an explicit numerical correctness comparison table between golden-model and RTL simulation outputs, and (3) FPGA synthesis, timing, and power results needed to complete the required results table and compute the Figure of Merit, as flagged throughout this report.

A Repository and File Reference

The following table maps report sections to source files in the team’s private GitHub repository (https://github.com/Mohaned-Waleed-Hosny/CNN_Accelerator), for reviewer traceability.

Table 6: Repository file map.

Repository Path	Used In
RTL/*.v	Sections 2–6
testbenches/*.v	Section 7
Golden_Model/run_golden_model.m	Section 7
Golden_Model/*.txt	Section 7 (test vectors)
Architecture Block Diagram/*.png	Section 2
TB_Pictures/*.png	Section 7
Scripts/run.do	Section 7 (simulation script)